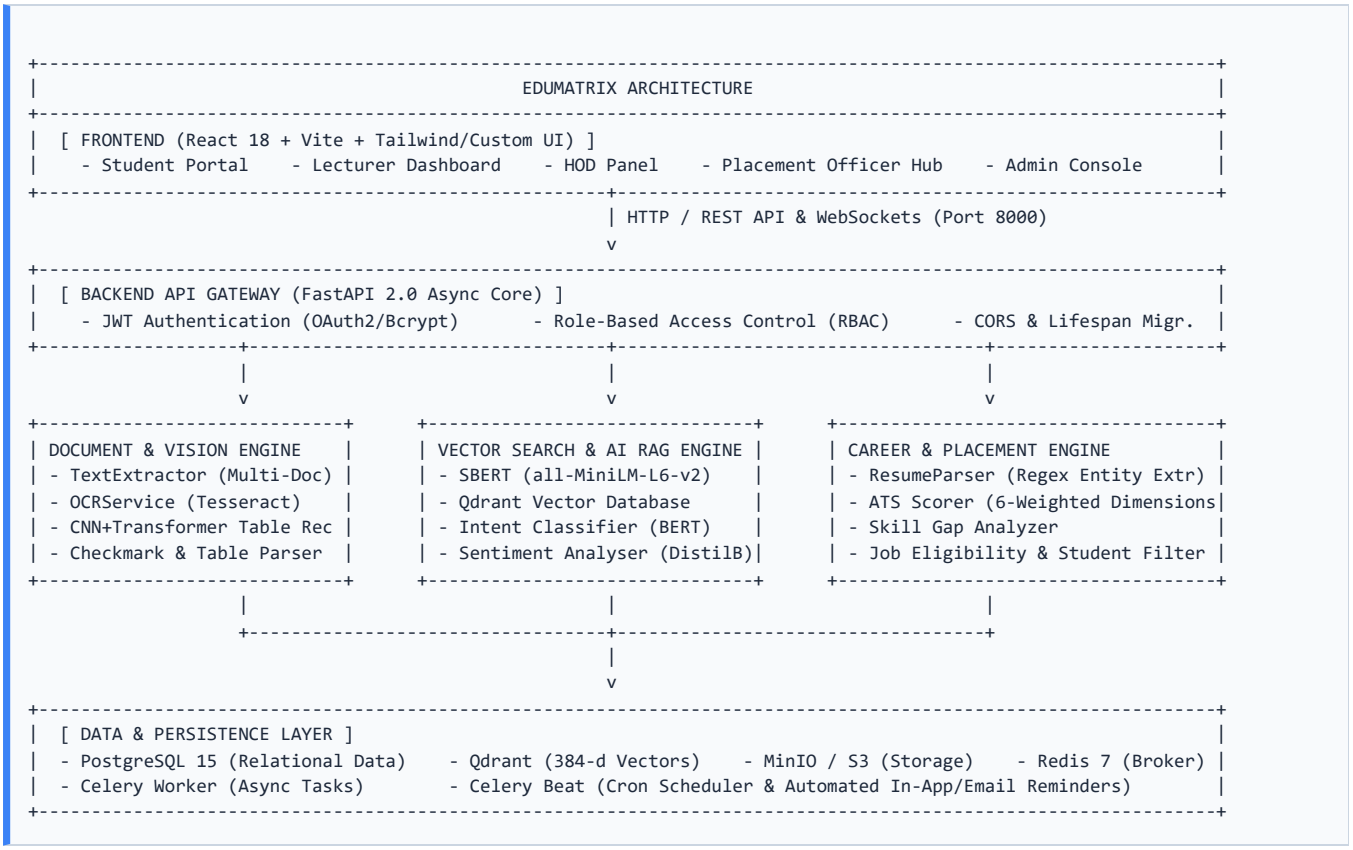
EDUMATRIX SYSTEM GUIDE

Institutional Academic & Placement Intelligence Platform

Comprehensive Technical Documentation • End-to-End Workflows • Every Single File • All ML Models

1. System Architecture Overview

Edumatrix is an enterprise-grade academic automation and placement intelligence platform. It bridges institutional operations (circular distribution, complex exam duty scheduling, role-based departmental communication) with career advancement intelligence (ATS resume parsing, skill gap analysis, job eligibility matching).



2. Machine Learning & Deep Learning Models Detailed

Edumatrix integrates 6 distinct AI/ML models tailored for specific visual, NLP, and structural tasks:

Model Name	Architecture / Tech	Dimensions / Parameters	Source / Weights Path	Primary Role in Edumatrix
Sentence-BERT (SBERT)	`all-MiniLM-L6-v2` Transformer	384-dimensional dense vectors	HuggingFace / `app/services/embedding_service.py`	Vectorizes circulars, policies, and queries for **Qdrant Vector RAG Search**.
Intent Classifier	Fine-tuned BERT Sequence Classifier	8 Output Classes (110M params)	`app/ml/intent_model_sbert/`	Classifies student messages into 8 intents (e.g., `ask_doubt`, `check_deadline`, `career_advice`).
Sentiment Analyser	Fine-tuned DistilBERT Classifier	3 Output Classes (Negative, Neutral, Positive)	`app/ml/sentiment_model/` (267MB weights)	Evaluates student queries and feedback tone with confidence probabilities.
Table Recognition Network	ResNet50 Backbone + 6-Layer Transformer Encoder + 3 Heads	9 Input Channels, 512 Hidden Dim (~145MB weights)	`app/ml/table_recognition/weights/best_model.pth`	Performs Multi-Task **Table Detection (TD)**, **Structure Recognition (TSR)**, and **OCR (TCR)** on complex/borderless tables.
Tesseract OCR	Bidirectional LSTM Neural Network (OEM 3)	Trained on English Language Glyphs	`tessdata/eng.traineddata` via `pytesseract`	Performs local text extraction on scanned document images and cropped table cells.
LayoutLMv3 (Optional VDU)	Multimodal Transformer (Text + Layout 2D + Image)	Visual Document Understanding	`app/ml/layoutlmv3/`	Spatial key-value extraction for structured institutional forms and certificates.

3. End-to-End System Workflows

Workflow A: User Authentication & Role-Based Access Control (RBAC)

Flow: User submits credentials at `/auth/login` → `security.verify_password()` compares bcrypt hash → `security.create_access_token()` generates signed JWT with User ID, Email, and Role → Client stores token in `localStorage` / `authStore.js` → FastAPI dependency `get_current_user` enforces endpoints based on role (`STUDENT` , `LECTURER` , `HOD` , `PLACEMENT_OFFICER` , `ADMIN`).

Workflow B: Circular Ingestion & Semantic Vector Indexing

Flow: 1. HOD/Admin uploads circular (PDF/DOCX/PNG) at `POST /files/upload` or `POST /circulars` .
2. File stream saved to **MinIO S3 Bucket** (`edumatrix-files`) via `MinioClient` .
3. Relational record created in **PostgreSQL** (`files` table).
4. Background Celery task triggers `TextExtractor` to extract text.
5. `EmbeddingService` encodes text into 384-d vector via SBERT.
6. `QdrantClient` upserts vector + metadata payload (`file_id` , `department` , `file_type`) to Qdrant collection `edumatrix_documents` .

Workflow C: Exam Duty Schedule Parsing & Automated Assignment

```
Upload Duty Timetable (PDF / Scanned Image)
   │
   ▼
ParserFactory.get_parser() ──► ImageParser / PDFParser
   │
   ▼
Image Preprocessing (Grayscale + CLAHE Contrast + Adaptive Thresholding)
   │
   ▼
OCR & Table Detection (Tesseract / Deep Learning Table Recognition Model)
   │
   ▼
CheckmarkDetector (Identifies ✓ / X marks in cells) + DateExtractor (Detects YYYY-MM-DD & FN/AN Sessions)
   │
   ▼
DutyMatcher (Fuzzy matching with PostgreSQL Lecturers in Department: removes Dr./Prof. titles & matches aliases)
   │
   ▼
DutyCreator (Inserts ExamDuty records in PostgreSQL)
   │
   ▼
DutyScheduler & Celery Beat (Dispatches automated in-app notifications & email reminders before exam dates)
```

Workflow D: Resume Parsing, ATS Scoring & Placement Matching

```
Student uploads Resume (PDF / DOCX) ──► ResumeParser extracts Contact Info, Skills, Education, Projects, Experience
   │
   ▼
ResumeAnalyzerService computes weighted ATS Score (0 - 100):
   ATS Score = 0.30(Skills) + 0.20(Education) + 0.20(Experience) + 0.15(Projects) + 0.10(Certifications) + 0.05(Format)
   │
   ▼
Skill Gap Analysis compares keywords against 100+ Skill Database (Languages, Frameworks, Cloud, Databases)
   │
   ▼
Company posts Job Drive (Min CGPA, Department, Allowed Backlogs, Required Skills)
   │
   ▼
PlacementService filters eligible candidates & triggers Celery placement notification tasks to students.
```

4. Complete File-by-File Technical Directory

A. Root Directory Files

File Path	What It Does	Where It Is Used / Dependencies
`docker-compose.yml`	Defines multi-container orchestration for PostgreSQL, Redis, Qdrant, MinIO, Backend, Celery Worker, Celery Beat, Frontend, and Nginx.	Root deployment entrypoint (`docker-compose up -d`).
`.env`	Holds environment variables (Database URL, JWT Secret, MinIO keys, Qdrant ports, Redis URL, CORS origins).	Loaded by `backend/app/core/config.py` and Docker Compose.
`test-chat.ps1`	Automated PowerShell test script for chat conversations and message delivery between users.	Integration testing for Chat subsystem.
`test-circulars.ps1`	PowerShell test suite verifying circular uploads, metadata parsing, and search endpoints.	Integration testing for Circular subsystem.
`test-notifications.ps1`	PowerShell test script validating Celery notification triggers and unread counter badges.	Integration testing for Notifications subsystem.

B. Backend Core (`backend/app/core/`)

File Path	What It Does	Where It Is Used / Dependencies
`core/config.py`	Loads Pydantic BaseSettings, database configurations, security parameters, MinIO, Qdrant, and Redis settings.	Imported across all backend services, database connections, and API routers.
`core/database.py`	Initializes SQLAlchemy Engine, `SessionLocal` sessionmaker, and `Base` ORM parent. Provides `get_db()` generator.	Used in all API endpoints with `db: Session = Depends(get_db)` .
`core/security.py`	Implements PassLib bcrypt password hashing, JWT token encoding (`create_access_token`), and decoding (`decode_access_token`).	Used in `api/auth.py` , `api/chat.py` , and all protected endpoints.

C. Backend Database Models (`backend/app/models/`)

File Path	What It Does	Where It Is Used / Relationships
`models/user.py`	Defines `User` table with roles (`STUDENT` , `LECTURER` , `HOD` , `PLACEMENT_OFFICER` , `ADMIN`), department, and hashed password.	Core identity table referenced by all foreign keys.
`models/circular.py`	Defines `Circular` table (title, content, department, file attachment, target audience, priority).	Used in `api/circulars.py` and circular search.
`models/exam_duty.py`	Defines `ExamDuty` table (lecturer_id, date, session, room/venue, subject_code, status, assigned_by).	Used in `services/exam_duty_processor.py` and `api/exam_duties.py` .
`models/duty_conflict.py`	Tracks schedule clashes (lecturer assigned to 2 rooms on the same date/session).	Used in `exam_duty_processor.py` conflict detection.
`models/file.py`	Stores uploaded file metadata (filename, MinIO bucket/object_path, file_size, mime_type, uploader_id, is_public).	Used in `services/file_storage/file_service.py` and `api/files.py` .
`models/file_version.py`	Tracks file revisions, version numbers, and update logs for uploaded circulars/materials.	Used in `components/FileVersionHistory.jsx` and file versioning API.
`models/chat.py`	Defines `ChatMessage` table (sender_id, receiver_id, content, is_read, timestamp).	Used in `api/chat.py` and WebSocket manager.
`models/notification.py`	Defines in-app notifications (user_id, title, message, type, is_read, reference_id).	Used in `api/notifications.py` and Celery tasks.
`models/reminder.py`	Defines scheduled alerts (event_type, trigger_time, user_id, status, channel: in-app/email).	Used in `services/reminder_scheduler.py` and `tasks/reminder_tasks.py` .
`models/profile.py`	Defines `StudentProfile` (USN, CGPA, department, semester, skills, backlogs, resume_url, placement_status).	Used in `api/profile.py` and placement filtering.
`models/resume.py`	Stores parsed resume data (skills, experience_years, education, ATS score, breakdown JSON, suggestions).	Used in `services/resume_analyzer_service.py` and `api/resume_analyzer.py` .
`models/certification.py`	Stores student certificates, issuing organization, issue date, and credential URL/file.	Used in `api/profile.py` and student verification.
`models/company.py`	Stores registered recruiting companies, website, industry, contact person, and placement drives.	Used in `api/placement.py` .
`models/job_posting.py`	Defines job openings (role, package/CTC, min_cgpa, allowed_depts, max_backlogs, deadline).	Used in `services/placement_service.py` and `api/placement.py` .
`models/application.py`	Tracks student job applications (student_id, job_id, resume_id, status: applied/shortlisted/rejected/selected).	Used in placement application tracking.
`models/shortlist.py`	Records placement officer shortlisting decisions for interview rounds.	Used in `api/placement.py` and interview scheduling.

D. Backend Schemas (`backend/app/schemas/`)

Defines Pydantic models for incoming request validation and JSON response serialization:

- `schemas/user.py`: `UserCreate`, `UserLogin`, `UserResponse`, `Token`.
- `schemas/circular.py`: `CircularCreate`, `CircularUpdate`, `CircularResponse`.
- `schemas/exam_duty.py`: `ExamDutyCreate`, `ExamDutyResponse`, `ExamDutyFilter`.
- `schemas/file.py`: `FileUploadResponse`, `FileSearchQuery`, `FileSearchResult`, `FileSearchResponse`.
- `schemas/chat.py`: `ChatMessageSend`, `ChatMessageResponse`, `ChatConversationResponse`.
- `schemas/notification.py`: `NotificationResponse`, `NotificationMarkRead`.
- `schemas/reminder.py`: `ReminderCreate`, `ReminderResponse`.
- `schemas/profile.py`: `ProfileCreate`, `ProfileUpdate`, `ProfileResponse`.
- `schemas/resume.py`: `AtsScoreResponse`, `SuggestionItem`, `ResumeResponse`.
- `schemas/placement.py`: `JobPostingCreate`, `JobPostingResponse`, `ApplicationResponse`.
- `schemas/certification.py`: `CertificationCreate`, `CertificationResponse`.

E. Backend API Routers (`backend/app/api/`)

Router File	Endpoint Prefix	Key Endpoints & Operations
`api/auth.py`	`/auth`	`POST /register`, `POST /login`, `GET /me` (JWT token issue & user profile resolution).
`api/circulars.py`	`/circulars`	`POST /` (Upload circular), `GET /` (List with department filters), `GET /search` (Vector search via Qdrant), `GET /{id}`.
`api/exam_duties.py`	`/exam-duties`	`POST /upload` (Trigger ExamDutyProcessor), `GET /my-duties` (Lecturer view), `GET /all` (HOD view), `PUT /{id}/status`.
`api/files.py`	`/files`	`POST /upload`, `GET /{id}/download` (MinIO presigned URLs), `POST /search` (Semantic Search), `DELETE /{id}`.
`api/chat.py`	`/chat`	`GET /users`, `GET /conversations`, `GET /messages/{user_id}`, `POST /send`, `WebSocket /ws/{user_id}`.
`api/ai.py`	`/ai`	`POST /classify-intent` (BERT Intent), `POST /analyse-sentiment` (DistilBERT Sentiment), `GET /health`.
`api/resume_analyzer.py`	`/resume`	`POST /upload` (Extract & calculate ATS score), `GET /my-resume`, `GET /suggestions`, `GET /ranking/{job_id}`.
`api/placement.py`	`/placement`	`POST /jobs`, `GET /jobs`, `POST /apply/{job_id}`, `GET /eligible-students/{job_id}`, `POST /shortlist`.
`api/profile.py`	`/profile`	`GET /me`, `PUT /me`, `POST /certifications`, `DELETE /certifications/{id}`.
`api/reminders.py`	`/reminders`	`GET /`, `POST /`, `PUT /{id}/dismiss` (In-app alerts management).
`api/notifications.py`	`/notifications`	`GET /`, `PUT /read-all`, `PUT /{id}/read` (Notification bell feeds).
`api/upload.py`	`/upload`	Generic multi-part file streaming endpoint to local uploads / MinIO.

F. Backend Services & Core Engines (`backend/app/services/`)

Service File	What It Does	Where It Is Used
`services/ocr_service.py`	Discovers Tesseract binary across Linux/Windows; extracts text from raw image bytes & scanned PDFs via `pdf2image`.	Used in `text_extractor.py` and image OCR pipelines.
`services/embedding_service.py`	Loads `all-MiniLM-L6-v2`; generates 384-d dense embeddings and calculates cosine similarity.	Used by Qdrant vector indexing and semantic search.
`services/intent_classifier.py`	Loads fine-tuned BERT intent model; classifies queries into 8 student intents with confidence scores.	Used in `api/ai.py` and AI chat assistant.
`services/sentiment_analyser.py`	Loads fine-tuned DistilBERT model; predicts negative/neutral/positive sentiment probabilities.	Used in `api/ai.py` for feedback and tone auditing.
`services/exam_duty_processor.py`	Master orchestrator: file parsing → table extraction → checkmark detection → teacher matching → database duty creation → scheduling reminders.	Used in `api/exam_duties.py`.
`services/duty_matcher.py`	Performs fuzzy matching on OCR extracted staff names against registered PostgreSQL lecturers, removing honorifics and matching initials.	Used in `exam_duty_processor.py`.
`services/duty_scheduler.py`	Calculates reminder offsets (24 hours and 1 hour before exam) and schedules notification entries.	Used in `exam_duty_processor.py` and `tasks/duty_tasks.py`.
`services/date_extractor.py`	Extracts dates in multiple formats (ISO `YYYY-MM-DD`, `DD/MM/YYYY`, `DD Month YYYY`) and maps FN/AN sessions.	Used in exam duty and circular parsing.
`services/resume_parser.py`	Parses resume PDFs/DOCXs using regex entity extractors for contact details, technical skills, degrees, and project headers.	Used in `resume_analyzer_service.py`.
`services/resume_analyzer_service.py`	Calculates 6-dimension weighted ATS score (0–	Used in `api/resume_analyzer.py`.

Service File	What It Does	Where It Is Used
	100), performs skill gap analysis against target tech stacks, and returns improvement tips.	
`services/placement_service.py`	Evaluates student eligibility (CGPA, departments, active backlogs, mandatory skills) for job postings and calculates candidate match score.	Used in `api/placement.py` and placement tasks.
`services/file_storage/minio_client.py`	Wraps MinIO / S3 SDK for creating buckets, uploading files, fetching presigned download URLs, and deleting objects.	Used in `services/file_storage/file_service.py`.
`services/file_storage/qdrant_client.py`	Manages Qdrant vector collection creation (384-d, Cosine, HNSW), vector upserts with payload, and nearest neighbor search.	Used in `search_service.py` and embedding pipelines.
`services/file_storage/text_extractor.py`	Comprehensive text extractor supporting PDF (pdfplumber/PyPDF2), DOCX, PPTX, XLSX, TXT, and Images (Tesseract).	Used in file upload, search indexing, and RAG.
`services/file_storage/search_service.py`	Executes hybrid search: vector similarity in Qdrant + relational filtering in PostgreSQL.	Used in `api/files.py` and `api/circulars.py`.
`services/file_storage/file_service.py`	Manages unified file upload flow: saves to MinIO → records in DB → triggers background text extraction and vector embedding.	Used in `api/files.py` and `api/upload.py`.

G. Backend Exam Duty Parser Subsystem (`backend/app/services/exam_duty_parser/`)

- `base_parser.py` : Abstract base class defining `parse()` and metadata extraction interface.
- `parser_factory.py` : Factory pattern returning `PDFParser` or `ImageParser` based on file extension.
- `pdf_parser.py` : Uses `pdfplumber` to extract text and tables with column/row layout preservation.
- `image_parser.py` : Preprocesses images (grayscale, contrast boost, adaptive threshold) and executes OCR.
- `table_parser.py` : Parses 2D table structures, column headers, dates, and assigns row/col indices.
- `checkmark_detector.py` : Uses image processing and pattern matching to detect tick marks (`✓` , `X`) in duty slots.
- `entity_mapper.py` : Maps parsed table cells into structured entities (Date, Session, Venue, Staff).
- `duty_creator.py` : Validates dates and inserts deduplicated `ExamDuty` records in PostgreSQL.

H. Backend Machine Learning Subsystem (`backend/app/ml/`)

- `train_intent.py` : Fine-tunes BERT on `training_data/intent_data.json` for 8 student intent classes.
- `train_intent_sbert.py` : Uses SentenceTransformers and cosine classifier for few-shot intent classification.
- `train_intent_setfit.py` : SetFit (Sentence Transformer Fine-Tuning) script for rapid few-shot training.
- `table_recognition/model.py` : PyTorch CNN (ResNet50) + 6-Layer Transformer multi-task table recognition model.
- `table_recognition/dataset.py` : Unified DataLoader fusing TabRecSet (TD & TSR) and Tencent OCR datasets.
- `table_recognition/trainer.py` : Multi-loss training loop with TensorBoard logging and `best_model.pth` checkpointing.
- `table_recognition/inference.py` : Production inference class running `TableRecognizer.predict()` .
- `table_recognition/config.py` : Hyperparameters, image dimensions (512x512 / 128x128), and batch sizes.
- `sentiment_model/` : DistilBERT weights (`model.safetensors`), tokenizer config, and `data/train.csv` (27k samples).

I. Backend Celery Asynchronous Tasks (`backend/app/tasks/`)

- `celery_app.py` : Configures Celery instance connected to Redis broker (`redis://localhost:6379/0`) with Celery Beat schedule.
- `duty_tasks.py` : Background task executing heavy OCR parsing and duty creation asynchronously.
- `file_tasks.py` : Background task computing text embeddings and upserting vectors to Qdrant without blocking API requests.
- `reminder_tasks.py` : Periodic cron job checking upcoming exam duties/circular deadlines and creating in-app notification rows.
- `placement_tasks.py` : Asynchronous batch notification dispatcher for new placement drives.

J. Backend WebSocket Real-Time Communication (`backend/app/websocket/`)

- `manager.py` : `ConnectionManager` storing active WebSocket connections (`user_id -> WebSocket`) to deliver instant real-time chat messages and notification badges.

K. Frontend Architecture (`frontend/src/`)

1. State Management (`frontend/src/store/`)

- `authStore.js` : Zustand store managing logged-in user state, JWT tokens, role flags, and logout actions.
- `fileStore.js` : Manages file browsing, uploaded files list, search filters, and download progress.
- `placementStore.js` : Manages active job drives, candidate eligibility status, and job applications.
- `resumeStore.js` : Stores ATS analysis results, skill radar charts, and improvement recommendations.

2. API Service Clients (`frontend/src/services/`)

- `api.js` : Axios instance configured with base URL `http://localhost:8000` and JWT Bearer token request interceptor.
- `fileService.js` : API calls for file upload, presigned MinIO downloads, and semantic vector search.
- `placementService.js` : API calls for job postings, application submissions, and candidate shortlisting.
- `resumeService.js` : API calls for resume upload, ATS score calculation, and ranking.

3. Key Frontend Components (`frontend/src/components/`)

Component Name	Visual & Functional Role
`Layout.jsx`	Master responsive application shell with dynamic Sidebar, Topbar, and Unified Notification Bell.
`ProtectedRoute.jsx`	Route guard checking JWT validity and role permissions before rendering protected pages.
`ChatWindow.jsx` & `ChatList.jsx`	Direct peer-to-peer real-time messaging interface with auto-scrolling, unread counts, and active chat user list.
`ExamDutyUpload.jsx` & `ExamDutyList.jsx`	HOD upload widget for exam duty PDFs/images with real-time extraction progress and lecturer assignment table.
`ExamDutyCard.jsx` & `ExamDutyConflict.jsx`	Lecturer dashboard card showing assigned date, session (FN/AN), venue, and schedule clash warnings.
`FileUpload.jsx` & `FileBrowser.jsx`	Drag-and-drop file uploader with category tagging, file previews, and folder browsing.
`FileSearch.jsx`	Semantic search bar querying Qdrant vectors with instant relevance ranking and department filters.
`ResumeUpload.jsx`	Interactive ATS resume scoring dashboard displaying 0–100 radial progress, skill tags, and improvement tips.
`SkillsManager.jsx` & `ProfileForm.jsx`	Student profile editor for USN, CGPA, department, semester, technical skills, and certifications.
`UnifiedBell.jsx` & `NotificationBell.jsx`	Real-time notification dropdown with unread badge count for exam duty alerts, circulars, and placement drives.

4. Frontend Role-Based Pages (`frontend/src/pages/`)

- `Landing.jsx` : Modern institutional landing page with feature showcases, login/register modals, and stats counters.
- `Login.jsx` & `Register.jsx` : Secure authentication pages with department, USN/Staff ID, and role selectors.
- `student/Dashboard.jsx` , `Circulars.jsx` , `Chat.jsx` , `Profile.jsx` : Student hub for notices, AI chat, and academics.
- `lecturer/` : Faculty portal displaying assigned exam duties, student inquiries, and circulars.

- `hod/` : Department Head panel for uploading duty schedules, managing faculty workload, and issuing notices.
- `placement/` : Placement Officer dashboard for creating job drives, evaluating student ATS scores, and shortlisting.
- `admin/` : Institutional admin console for managing user roles, department directories, and system health.
